Short Strings Matter More for State Merging Algorithms than NNs

Anonymous ACL submission

1 Introduction

This paper compares the results of a state merging algorithm and four neural network (NN) architectures on an extension of MLREGTEST, a benchmark for learning subregular language classes relevant to natural language (van der Poel et al., 2024). The MLREGTEST data contains only strings of length 20-29, but Soubki and Heinz (2023) demonstrated that augmenting a preliminary version with shorter strings improved performance of the state merging algorithm. We replicate Soubki and Heinz and van der Poel et al. on the final version of MLREGTEST, comparing performance of a state merging algorithm with four NNs trained on the original MLREGTEST data, this data augmented with short strings, and data containing *only* short strings. We find that the state merging algorithm benefits greatly from short strings, while the neural networks rely more on long strings.

2 Learning Systems

2.1 State-Merging Systems

Following Soubki and Heinz (2023), we use the state-merging software FLEXFRINGE (Verwer and Hammerschmidt, 2025),[1] which implements the EDSM (Lang et al., 1998) and RPNI (Oncina et al., 1992) algorithms.[2] Both algorithms construct a prefix tree from their input and search for pairs of states to merge; RPNI does so in a breadth-first fashion, while EDSM computes a score for all possible pairs and attempts to merge the pair with the highest score.[3]

FLEXFRINGE provides several parameters which impact learning behavior. To find the best model, we conducted a limited gridsearch over the following subset:[4] `PerformFirst` (EDSM vs. RPNI), `Extend` (color blue states red that cannot be merged with any red), `ShallowFirst` (consider coloring blue states closest to the root first), and `SearchStrategy` (searchdeep, searchlocal, searchglobal, searchpartial, or none). Grid searching was done on the small partition of the PLUSSHORT data using overall accuracy on the development set (see §3). The best performing model, used in the remainder of the paper, was `PerformFirst = 0` (i.e., RPNI), `Extend = 1`, `ShallowFirst = 0`, and `SearchStrategy = SearchDeep`.

2.2 Neural Systems

Following van der Poel et al. (2024), we test four neural architectures, all consisting of a trainable embedding layer followed by a unidirectional recurrent module (either a **SIMPLE** RNN, a **GRU**, or an **LSTM**) or two consecutive **TRANSFORMER** blocks, dense feed-forward layers, dropout, layer normalization, and softmax activation. All NNs use the same hyperparameters as van der Poel et al..[5]

3 Evaluation

MLREGTEST (van der Poel et al., 2024) contains training, development, and test data from 1,800 regular languages; we follow Soubki and Heinz (2023) in excluding languages with alphabet sizes of 64 due to unicode errors, thus considering 1,140. The strings in MLREGTEST are of length 20-29, balanced evenly between positive and negative examples. Soubki and Heinz found that augmenting

[1] We use the same version of FLEXFRINGE used by Soubki and Heinz: commit `4d1449c` of the software available here.

[2] We exclude ALERGIA from consideration because of the infeasibly high compute demands and comparatively low performance noted by Soubki and Heinz.

[3] Notably, the implementation of RPNI in FLEXFRINGE differs slightly from that in Oncina et al. (1992): while Oncina et al. merged states in length-lexicographic order when ties arise, FLEXFRINGE instead calculates a score, akin to EDSM.

[4] We held several other parameters at their default state because modifying them led to errors in the `FlexFringe` executable. These are: `ReverseTraces`, `DepthCheck`, `SymbolCheck`, `SinksOn`, `SinkCount`, and `MergeScore`.

[5] Implementations of the neural networks are available at: https://github.com/heinz-jeffrey/subregular-learning/

Model	OS	PlusShort			OnlyLong		
		S	M	L	S	M	L
FF	.709	.747	.775	-	.534	.560	-
Simple	.679	.732	.782	.837	.720	.774	.832
GRU	**.792**	**.906**	**.957**	.965	**.847**	**.942**	.949
LSTM	.646	.727	.870	.926	.638	.837	.912
Trans.	.709	.762	.811	.857	.754	.806	.806

Table 1: F1 score by model.

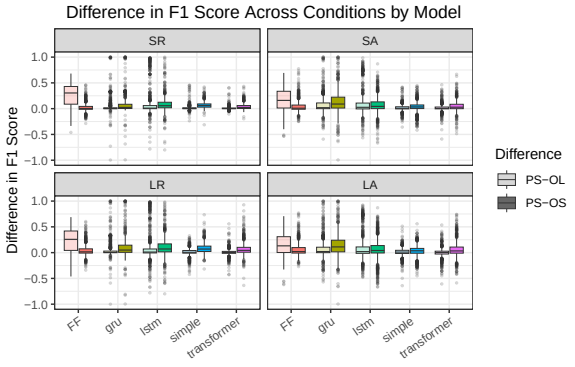

Figure 1: Difference in F1 between PlusShort and other training setups by model & test type.

train with roughly 1,000 short strings (length 1-19) significantly improved FlexFringe performance despite sacrificing the balance of positive and negative examples. We build on Soubki and Heinz by considering three training setups:

OnlyShort: only strings of length 1-19. For each length, 25 positive (negative) strings were sampled with replacement. If no positive (negative) strings of a given length existed, none were drawn.

PlusShort: MLRegTest augmented with short strings, replicating Soubki and Heinz (2023).

OnlyLong: original MLRegTest data.

MLRegTest contains four test sets per language, based on the length of the strings (Short = 20-29, Long = 31-50) and the method of generation (Random, sampled randomly without replacement or Adversarial, with paired positive and negative examples with an edit distance of 1).

4 Results

Table 1 gives the F1 score of each model, averaged across languages and test types[6]. Though the GRU consistently outperforms the other models, its performance decreases far more between PlusShort and OnlyShort than FlexFringe (Figure 1), which outperforms all other NNs on OnlyShort.

[6]FlexFringe models for the large partition are still running at time of submission but will be completed for camera-ready and do not bear on our key takeaways.

Similarly, FlexFringe sees the largest jump in performance from OnlyLong to PlusShort, also illustrated in Figure 1. Taken together, these results suggest that FlexFringe benefits far more from the inclusion of short strings, while the neural models — particularly the GRU — rely more on long strings to achieve their performance.

5 Conclusions

The strongest model overall was the GRU, which performed best across partitions. Strikingly, however, the performance of the state merging algorithm benefitted far more from the addition of short strings than that of any of the neural models: it performed most competitively on OnlyShort. Meanwhile, increasing the size of the training set was less helpful for the state merging algorithm than for the neural models. This shows a certain attunement of the state merging algorithm to the kinds of input samples that are relevant for learning natural languages – i.e., short and small.

Future research could expand the gridsearch (§2.1) to cover additional parameters provided by FlexFringe, which were omitted from this study due to bugs in the version of the library used here.

References

Kevin J Lang, Barak A Pearlmutter, and Rodney A Price. 1998. Results of the abbadingo one dfa learning competition and a new evidence-driven state merging algorithm. In *International Colloquium on Grammatical Inference*, pages 1–12. Springer.

José Oncina, Pedro Garcia, and 1 others. 1992. Inferring regular languages in polynomial update time. *Pattern recognition and image analysis*, 1(49-61):10–1142.

Adil Soubki and Jeffrey Heinz. 2023. Benchmarking state-merging algorithms for learning regular languages. In *Proceedings of 16th edition of the International Conference on Grammatical Inference*, volume 217 of *Proceedings of Machine Learning Research*, pages 181–198. PMLR.

Sam van der Poel, Dakotah Lambert, Kalina Kostyszyn, Tiantian Gao, Rahul Verma, Derek Andersen, Joanne Chau, Emily Peterson, Cody St. Clair, Paul Fodor, Chihiro Shibata, and Jeffrey Heinz. 2024. Mlregtest: A benchmark for the machine learning of regular languages. *Journal of Machine Learning Research*, 25(283):1–45.

Sicco Verwer and Christian Hammerschmidt. 2025. Flexfringe: Modeling software behavior by learning probabilistic automata. *Logical Methods in Computer Science*, 21.